



PROJECT REPORT

ORANGE PROJECT

DONE

Theia

Author(s):

Famura

Contents

1.	Introduction.....	4
2.	Desktop.....	4
2.1.	Project Technologies	4
2.1.1.	Module(s)	4
2.1.2.	Coding Language(s)	5
2.1.3.	Libraries & External Additions	5
2.2.	Driver.....	6
2.2.1.	Driver Entry Point	6
2.2.2.	Server Bootstrap.....	7
2.2.3.	Virtual Headset.....	9
2.2.4.	Virtual Controller	12
2.2.5.	Display Redirection.....	13
2.2.6.	Frame Encoding	14
2.3.	Installer	15
2.3.1.	Installer Entry Point and UI Setup	15
2.3.2.	Installer Flow and Step System.....	15
2.3.3.	SteamVR Detection and Validation	17
2.3.4.	Default Install Path and Package Source	18
2.3.5.	Actual Install Process.....	18
2.3.6.	UI Controls & Event Handling	19
2.4.	Host.....	20
2.4.1.	Entry Point	20
2.4.2.	Host Orchestration	20
2.4.3.	Renderer	22
2.4.4.	Debug & Control UI	24
2.4.5.	Streaming Handoff	25
2.4.6.	Pose and Tracking State.....	26
3.	Mobile	27
3.1.	Project Technologies	27
3.1.1.	Module(s)	27
3.1.2.	Coding Language(s)	27
3.1.3.	Libraries & External Additions	27
3.2.	Application Shell.....	28
3.3.	Session Coordination	28
3.4.	Video Reception	29

3.5.	Video Decoding	29
3.6.	Pose Transmission	30
3.7.	Protocol Layer	31
4.	Project Analysis.....	31
4.1.	Test	31
4.2.	Upgradeable Things	31
5.	Project Comments	31
5.1.	To-Do's When Using.....	31
5.2.	Creator Comments	32
6.	Credits.....	32
7.	References	32
8.	Disk Contents	33

1. Introduction

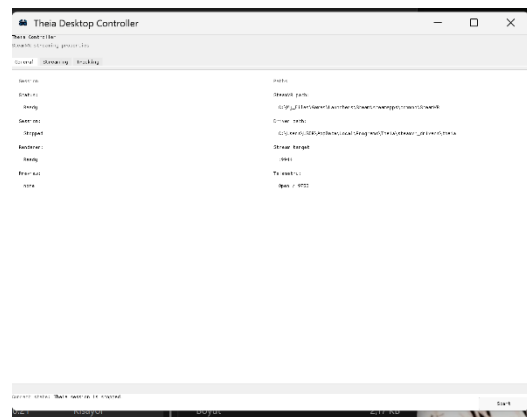
Theia is a two-part real-time VR streaming project built around a Windows desktop host and an Android mobile client. The main idea of the project is to move a live visual feed from the desktop side to the phone over the network, while also keeping the mobile device's orientation data available as part of the overall interaction loop. In other words, Theia is not just a video sender or a simple mobile viewer. It is a small streaming system with platform-specific responsibilities, custom networking logic, runtime coordination, and VR-oriented integration.

What makes the project interesting is that the two sides are designed to complement each other instead of duplicating work. The desktop application is responsible for launching and managing the host runtime, preparing the stream, and handling the Windows and SteamVR side of the workflow. The mobile application is responsible for receiving the stream, decoding it efficiently, displaying it on screen, and collecting motion data from the device sensors. That separation keeps the project modular and makes the flow easier to understand and extend.

The project code is organized so that each platform handles the tasks it is best suited for. The Windows side deals with system integration, rendering, encoding, UDP transport, and VR runtime management. The Android side deals with surface rendering, hardware decoding, and sensor-driven pose capture. This architecture makes Theia feel like a practical system rather than a demo built around one single feature.

2. Desktop

The desktop application is the host side of Theia. This is where the runtime starts, where the stream is prepared, and where the SteamVR integration is managed. The desktop project is written in C++ and uses a CMake-based build system. From the code, the desktop side is clearly built as the control center of the entire project. The application entry point is in `Desktop/main.cpp`, and the runtime orchestration is handled by `Desktop/src/App/Application.cpp`.



The desktop code pulls together a number of responsibilities that would normally be split across different tools or services. It initializes networking, reads configuration, prepares the VR runtime, opens the render window, and manages the streaming session. That means the desktop side is not only “sending video”; it is also acting as the control environment that decides when the session starts, when it stops, how the driver is registered, and which host address the mobile client should use.

2.1. Project Technologies

2.1.1. Module(s)

- **Windows (Windows 11):**

The entire project is designed to run in Windows systems. Did not tested in lower Windows versions like Windows 7.

2.1.2. Coding Language(s)

- **C++ (C++20 Standard) – CLion & CMake:**

The entire project is written in C++ and is set to compile with the C++20 standard. This provides modern language features and compatibility with CLion. Clion is used because of familiarity with IntelliJ's IDE structure and easier git usage.

2.1.3. Libraries & External Additions

- **Winsock2.h**
Provides the Windows Sockets API used for UDP/TCP network communication.
- **ws2tcpip.h**
Provides modern socket helper functions such as `inet_pton`, which is used for IP address handling.
- **windows.h**
Provides the core Windows API used for process creation, window/runtime integration, and system-level calls.
- **FFmpeg (libavcodec, libavutil, libswscale, libavformat, libavdevice, libavfilter, swresample)**
Used for frame encoding, pixel format conversion, and video streaming preparation.
- **x264**
Used as part of the H.264 encoding pipeline when static FFmpeg linking is enabled.
- **glad**
Used to load OpenGL function pointers at runtime.
- **glfw3**
Used to create the desktop window, OpenGL context, and handle input/event polling.
- **glm**
Used for vector and math operations in rendering and pose-related calculations.
- **imgui**
Used for the debug overlay and developer-facing runtime UI.
- **spdlog**
Used for structured logging across the desktop application.
- **stb / stb_image**
Used for lightweight image loading and related asset handling.
- **OpenVR SDK**
Used for SteamVR driver integration and VR runtime communication.
- **CMake**
Used as the build system and project configuration layer.
- **SteamVR executable tools (vrpathreg.exe, vrstartup.exe)**
Used to register the custom driver and launch the SteamVR runtime.
- **CreateProcessA / taskkill.exe**
Used for starting and stopping external runtime processes from the host application.
- **d3d11, dxgi**
Used in the SteamVR driver side for DirectX and graphics-related integration.
- **mfplat, mfuuid, wmcodecdspuuid**
Used for multimedia and codec-related driver/runtime support.
- **ole32, shell32, shlwapi, user32, bcrypt, crypt32, ncrypt, psapi, rpcrt4, secur32, comdlg32, vfw32, strmiids, usp10**
Used by the desktop host, installer, and SteamVR driver for Windows subsystem functionality.

2.2. Driver

The driver part of Theia is the bridge between the desktop application and SteamVR. This is the layer that makes the project feel like a real VR integration instead of a normal video streaming app. SteamVR does not directly know about Theia's application logic; instead, it loads the custom driver module and communicates with the driver through the OpenVR server interfaces.

The driver is structured into several parts:

- HmdDriverFactory.cpp exposes the driver entry point to SteamVR.
- TheiaServerProvider.cpp creates and registers the virtual devices.
- TheiaHmdDriver.cpp represents the virtual headset.
- TheiaControllerDriver.cpp represents the virtual controller.
- TheiaDisplayRedirectDriver.cpp captures the SteamVR render output.
- TheiaFrameEncoder.cpp converts and streams the captured image data.

This split is important because each file has one specific responsibility. SteamVR sees them as driver objects, but internally they are cooperating parts of one system.

2.2.1. Driver Entry Point

The first thing SteamVR looks for is the exported factory function. That is implemented in Desktop/drivers/theia/src/HmdDriverFactory.cpp. This is the point where the driver becomes visible to SteamVR.

This file is very small, but it is the starting point of the whole driver. SteamVR loads the DLL, calls HmdDriverFactory, and asks for the IServerTrackedDeviceProvider interface. If the requested interface name matches, the function returns the global TheiaServerProvider instance. In simple terms, this file is the door that SteamVR uses to enter Theia's driver code.

What happens here:

- SteamVR loads the driver DLL.
- It asks for the server provider interface.
- The factory returns the Theia provider object.
- After that, SteamVR can initialize the driver normally.

```
THEIA_HMD_DLL_EXPORT void* HmdDriverFactory(const char* interface_name, int* return_code) {  
    if (std::strcmp(interface_name, vr::IServerTrackedDeviceProvider_Version) == 0) {  
        if (return_code != nullptr) {  
            *return_code = vr::VRInitError_None;  
        }  
        return &g_server_provider;  
    }  
  
    if (return_code != nullptr) {  
        *return_code = vr::VRInitError_Init_InterfaceNotFound;  
    }  
    return nullptr;  
}
```

2.2.2. Server Bootstrap

The next important piece is Desktop/drivers/theia/src/TheiaServerProvider.cpp. This is the layer that SteamVR calls after the factory returns the provider instance. The Init function is where the driver actually starts setting itself up. It receives the SteamVR driver context and then creates three main objects: TheiaHmdDriver, TheiaControllerDriver and TheiaDisplayRedirectDriver.

```
VR_INIT_SERVER_DRIVER_CONTEXT(driver_context);

vr::VRDriverLog()->Log("Theia server provider init started");

display_redirect_ = std::make_unique<TheiaDisplayRedirectDriver>();
hmd_ = std::make_unique<TheiaHmdDriver>();
controller_ = std::make_unique<TheiaControllerDriver>(*hmd_);
hmd_->SetVirtualDisplay(static_cast<vr::IVRVirtualDisplay*>(display_redirect_.get()));
hmd_->SetControllerDriver(controller_.get());

const bool added = vr::VRServerDriverHost()->TrackedDeviceAdded(hmd_->SerialNumber().c_str(),
                                                                vr::TrackedDeviceClass_HMD,
                                                                hmd_.get());
```

After creating them, the provider connects them together:

- the HMD receives the virtual display pointer

```
if (!added) {
    vr::VRDriverLog()->Log("Theia failed to add virtual HMD");
    hmd_.reset();
    return vr::VRInitError_Driver_Unknown;
}

char hmd_message[192] = {};
std::snprintf(hmd_message,
              sizeof(hmd_message),
              "Theia virtual HMD queued for activation serial='%s'",
              hmd_->SerialNumber().c_str());
vr::VRDriverLog()->Log(hmd_message);
```

- the HMD receives the controller driver pointer

```
if (!controller_added) {
    vr::VRDriverLog()->Log("Theia failed to add controller device");
    controller_.reset();
    display_redirect_.reset();
    hmd_.reset();
    return vr::VRInitError_Driver_Unknown;
}

char controller_message[224] = {};
std::snprintf(controller_message,
              sizeof(controller_message),
              "Theia controller queued for activation serial='%s'",
              controller_->SerialNumber().c_str());
vr::VRDriverLog()->Log(controller_message);
```

- then each device is registered with `VRServerDriverHost()->TrackedDeviceAdded(...)`

```
const bool controller_added =
    vr::VRServerDriverHost()->TrackedDeviceAdded(controller_->SerialNumber().c_str(),
                                                  vr::TrackedDeviceClass_Controller,
                                                  controller_.get());
```

This is a very important part of the driver architecture. SteamVR expects tracked devices to be registered through the server host interface, and Theia uses that mechanism to present itself as a virtual headset plus related device.

The `RunFrame` function then forwards frame updates to all three device objects. This is how the driver keeps itself alive during runtime. Each frame firstly updates the HMD pose, then updates controller's pose and input state, then continues with updating display redirect processes rendering events and frame capture, lastly finishes with provider also polls SteamVR events and dispatches them.

```
void TheiaServerProvider::RunFrame() {
    if (hmd_ != nullptr) {
        hmd_->RunFrame();
    }
    if (controller_ != nullptr) {
        controller_->RunFrame();
    }
    if (display_redirect_ != nullptr) {
        display_redirect_->RunFrame();
    }

    vr::VREvent_t event{};
    while (vr::VRServerDriverHost()->PollNextEvent(&event, sizeof(event))) {
        if (hmd_ != nullptr) {
            hmd_->ProcessEvent(event);
        }
        if (controller_ != nullptr) {
            controller_->ProcessEvent(event);
        }
        if (display_redirect_ != nullptr) {
            display_redirect_->ProcessEvent(event);
        }
    }
}
```

So, this file is basically the traffic controller for the whole driver. At first look this approach and this adaptation might seem a little costly but it runs smoothly with good optimization. So, yeah.

2.2.3. Virtual Headset

The HMD side is the heart of the driver. The main file is Desktop/drivers/theia/src/TheiaHmdDriver.cpp, with the interface defined in Desktop/drivers/theia/src/TheiaHmdDriver.h.

This class represents the virtual headset that SteamVR thinks it is talking to. It is responsible for reporting headset pose, display configuration, and interaction state.

2.2.3.1. What Happens During Activation

When SteamVR activates the HMD, the driver resets its internal pose state, fills SteamVR device properties, declares itself as an HMD, sets model/manufacture/tracking metadata, sets display-related values like IPD and refresh rate, optionally opens a UDP socket for phone pose data. This part is done in Activate(...). It is where the headset becomes a valid VR tracked device from SteamVR's point of view. It is where the headset becomes a valid VR tracked device from SteamVR's point of view.

```
vr::EVRInitError TheiaHmdDriver::Activate(const std::uint32_t object_id) {  
    object_id_ = object_id;  
    frame_number_ = 0;  
    pose_update_count_ = 0;  
    {  
        std::lock_guard<std::mutex> lock(pose_mutex_);  
        keyboard_pose_ = {};  
    }  
    phone_rotation_raw_ = IdentityQuaternion();  
    phone_rotation_offset_ = IdentityQuaternion();  
    has_phone_rotation_ = false;  
    reset_key_was_down_ = false;  
    select_key_was_down_ = false;  
    back_key_was_down_ = false;  
    interaction_active_ = false;  
    proximity_pressed_ = false;  
    has_last_phone_pose_time_ = false;  
    has_last_keyboard_update_time_ = false;  
    last_phone_pose_sequence_ = 0;  
    duplicate_phone_pose_packets_ = 0;  
    stale_phone_pose_packets_ = 0;  
    is_active = true;  
}
```

The code also sets properties such as: tracking system name / model number / manufacturer / display frequency / head-to-eye depth / IMU or proximity-related flags

These properties matter because SteamVR uses them to understand how the device should behave.

```

const vr::PropertyContainerHandle_t container = vr::VRProperties()->TrackedDeviceToPropertyContainer(object_id);
vr::VRProperties()->SetInt32Property(container, vr::Prop_DeviceClass_Int32, vr::TrackedDeviceClass_HMD);
vr::VRProperties()->SetStringProperty(container, vr::Prop_TrackingSystemName_String, kDefaultTrackingSystem);
vr::VRProperties()->SetStringProperty(container, vr::Prop_ModelNumber_String, model_number.c_str());
vr::VRProperties()->SetStringProperty(container, vr::Prop_ManufacturerName_String, kDefaultManufacturer);
vr::VRProperties()->SetStringProperty(container, vr::Prop_RenderModelName_String, model_number.c_str());
vr::VRProperties()->SetFloatProperty(container, vr::Prop_UserIpDmeters_Float, display_config.ipd_meters);
vr::VRProperties()->SetFloatProperty(container, vr::Prop_DisplayFrequency_Float,
    display_config.display_frequency_hz);
vr::VRProperties()->SetFloatProperty(container, vr::Prop_UserHeadToEyeDepthMeters_Float,
    display_config.head_to_eye_depth_meters);
vr::VRProperties()->SetFloatProperty(container, vr::Prop_SecondsFromVsyncToPhotons_Float,
    display_config.vsync_to_photons_seconds);
vr::VRProperties()->SetUint64Property(container, vr::Prop_CurrentUniverseId_Uint64, 1);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_IsOnDesktop_Bool, false);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_DisplayDebugMode_Bool, true);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_HasDisplayComponent_Bool, true);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_HasVirtualDisplayComponent_Bool,
    virtual_display_ != nullptr);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_ReportsTimeSinceVSync_Bool, true);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_NeverTracked_Bool, false);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_DeviceHasNoIMU_Bool, true);
vr::VRProperties()->SetBoolProperty(container, vr::Prop_ContainsProximitySensor_Bool, true);

```

2.2.3.2. How Pose is Produced

The HMD pose comes from two possible sources the phone orientation data received over UDP and keyboard-controlled fallback motion.

The phone pose path is especially important. The driver opens a UDP socket on port 9955 and listens for incoming pose packets from the Android app. When packets arrive the driver checks the packet header and then ignores duplicates and stale sequence numbers. This allows it to normalize the quaternion so it can store the latest phone rotation. And it also records the time of the last valid packet. That means the phone is not just a viewer device. It also contributes orientation input back to the SteamVR side.

```

vr::DriverPose_t TheiaHmdDriver::GetPose() {
    const std::uint64_t frame = frame_number_.load();
    const double bob = std::sin(static_cast<double>(frame) * 0.01) * 0.015;
    KeyboardPoseState keyboard_pose{};
    {
        std::lock_guard<std::mutex> lock(pose_mutex_);
        keyboard_pose = keyboard_pose_;
    }

    vr::DriverPose_t pose{};
    pose.qWorldFromDriverRotation = IdentityQuaternion();
    pose.qDriverFromHeadRotation = IdentityQuaternion();
    pose.qRotation = has_phone_rotation_
        ? NormalizeQuaternion(MultiplyQuaternion(phone_rotation_offset_, phone_rotation_raw_))
        : EulerToQuaternion(
            keyboard_pose.pitch_degrees,
            keyboard_pose.yaw_degrees,
            keyboard_pose.roll_degrees);
    pose.vecPosition[0] = keyboard_pose.x;
    pose.vecPosition[1] = keyboard_pose.y + bob;
    pose.vecPosition[2] = keyboard_pose.z;
    pose.result = vr::TrackingResult_Running_OK;
    pose.poseIsValid = true;
    pose.deviceIsConnected = true;
    pose.shouldApplyHeadModel = true;
    return pose;
}

```

```

if (IsKeyDown('W')) {
    keyboard_pose_.x += forward_x * move_distance;
    keyboard_pose_.z += forward_z * move_distance;
}
if (IsKeyDown('S')) {
    keyboard_pose_.x -= forward_x * move_distance;
    keyboard_pose_.z -= forward_z * move_distance;
}
if (IsKeyDown('A')) {
    keyboard_pose_.x -= right_x * move_distance;
    keyboard_pose_.z -= right_z * move_distance;
}
if (IsKeyDown('D')) {
    keyboard_pose_.x += right_x * move_distance;
    keyboard_pose_.z += right_z * move_distance;
}

```

If phone pose is unavailable or user are having problem with it, the driver falls back to keyboard navigation. The code supports some basic keyboard functions. W, A, S, D for movement; Q, E for vertical movement; arrow keys for looking around; R for reset; Enter and Backspace for interaction events. So the HMD can still be tested without the phone connection. R for reset and Enter / Backspace functions are currently working even with phone connection so that user can have an easier time when using the app. Otherwise adjusting natural pose might be a nightmare.

2.2.3.3. What Happens Every Frame

In RunFrame(), the HMD: increments the frame counter, polls phone pose packets, polls keyboard input state, updates the fallback keyboard pose, builds the final VR pose and lastly sends it to SteamVR using TrackedDevicePoseUpdated.

```

void TheiaHmdDriver::RunFrame() {
    ++frame_number_;
    const std::uint32_t object_id = object_id_.load();
    if (is_active_ && object_id != vr::k_unTrackedDeviceIndexInvalid) {
        const auto now = std::chrono::steady_clock::now();
        const double delta_seconds = has_last_keyboard_update_time_
            ? std::chrono::duration<double>(now - last_keyboard_update_time_).count()
            : 0.0;

        last_keyboard_update_time_ = now;
        has_last_keyboard_update_time_ = true;
        PollPhonePose();
        PollKeyboardActions();
        UpdateKeyboardPose(delta_seconds);

        const vr::DriverPose_t pose = GetPose();
        vr::VRServerDriverHost()->TrackedDevicePoseUpdated(object_id, pose, sizeof(vr::DriverPose_t));

        const std::uint64_t pose_count = ++pose_update_count_;
        if (pose_count == 1 ||
            (trace_pose_updates_ && pose_trace_interval_ > 0 && pose_count % pose_trace_interval_ == 0)) {
            char message[256] = {};
            std::snprintf(message,
                sizeof(message),
                "Theia HMD pose submitted objectId=%u count=%llu valid=%d connected=%d y=%.3f result=%d",
                object_id,
                static_cast<unsigned long long>(pose_count),
                static_cast<int>(pose.poseIsValid),
                static_cast<int>(pose.deviceIsConnected),
                pose.vecPosition[1],
                static_cast<int>(pose.result));
            Log(message);
        }
    }
}

```

This is the function that keeps the virtual headset alive in the SteamVR runtime. If you think of SteamVR as the consumer, RunFrame is the place where Theia continuously feeds it updated headset state.

2.2.3.4. Why This Part Matters?

This class is the part that makes Theia feel like a real tracked headset. It is not only a display target; it also owns pose, input, and interaction state. That is why the HMD file is one of the most important pieces of the project and this report.

2.2.4. Virtual Controller

The controller driver is smaller than the HMD, but it is still important. It lives in Desktop/drivers/theia/src/TheiaControllerDriver.cpp. This class exists to represent a right-hand controller inside SteamVR. It is not a full physical controller implementation in the hardware sense. Instead, it is a virtual controller object that follows the HMD's pose and can emit button events.

2.2.4.1. What It Does

On activation, the controller: registers itself as a SteamVR controller, sets its tracking system and model metadata, declares its input profile, marks itself as the right-hand controller and exposes supported buttons like trigger and application menu. This makes SteamVR treat it like a usable controller device.

```
vr::EVRInitError TheiaControllerDriver::Activate(const std::uint32_t object_id) {
    object_id = object_id;
    is_active_ = true;

    const vr::PropertyContainerHandle_t container = vr::VRProperties()->TrackedDeviceToPropertyContainer(object_id);
    vr::VRProperties()->SetInt32Property(container, vr::Prop_DeviceClass_Int32, vr::TrackedDeviceClass_Controller);
    vr::VRProperties()->SetStringProperty(container, vr::Prop_TrackingSystemName_String, kControllerTrackingSystem);
    vr::VRProperties()->SetStringProperty(container, vr::Prop_ModelNumber_String, kControllerModelNumber);
    vr::VRProperties()->SetStringProperty(container, vr::Prop_ManufacturerName_String, kControllerManufacturer);
    vr::VRProperties()->SetStringProperty(container, vr::Prop_RenderModelName_String, kControllerRenderModel);
    vr::VRProperties()->SetStringProperty(container, vr::Prop_ControllerType_String, kControllerType);
    vr::VRProperties()->SetStringProperty(container, vr::Prop_InputProfilePath_String, kControllerInputProfile);
    vr::VRProperties()->SetInt32Property(container, vr::Prop_ControllerRoleHint_Int32, vr::TrackedControllerRole_RightHand);
    vr::VRProperties()->SetBoolProperty(container, vr::Prop_HasControllerComponent_Bool, true);
    vr::VRProperties()->SetBoolProperty(container, vr::Prop_NeverTracked_Bool, false);
    vr::VRProperties()->SetUInt64Property(
        container,
        vr::Prop_SupportedButtons_UInt64,
        vr::ButtonMaskFromId(vr::k_EButton_SteamVR_Trigger) |
        vr::ButtonMaskFromId(vr::k_EButton_ApplicationMenu));
    vr::VRProperties()->SetInt32Property(container, vr::Prop_Axis0Type_Int32, vr::k_eControllerAxis_None);
    vr::VRProperties()->SetInt32Property(container, vr::Prop_Axis1Type_Int32, vr::k_eControllerAxis_Trigger);

    char message[192] = {};
    std::snprintf(message,
        sizeof(message),
        "Theia controller activated objectId=%u serial='%s'",
        object_id,
        serial_number_.c_str());
    Log(message);
    return vr::EVRInitError_None;
}
```

2.2.4.2. Pose Behavior

The controller's pose is derived from the HMD's tracking snapshot.

It takes the headset position and rotation, applies a local offset, and rotates that offset with the headset orientation. That way, the controller appears to live in the same space as the headset rather than floating independently. So this part is basically: "follow the headset, but behave like a separate tracked object."

2.2.4.3. Button Events

The controller can also emit SteamVR button events using `SetSelectPressed(...)` maps to the trigger and `SetBackPressed(...)` maps to the application menu button.

```
void TheiaControllerDriver::SetSelectPressed(const bool pressed) {
    if (pressed == select_pressed_) {
        return;
    }
    select_pressed_ = pressed;
    EmitButtonEvent(vr::k_EButton_SteamVR_Trigger, pressed);
}

void TheiaControllerDriver::SetBackPressed(const bool pressed) {
    if (pressed == back_pressed_) {
        return;
    }
    back_pressed_ = pressed;
    EmitButtonEvent(vr::k_EButton_ApplicationMenu, pressed);
}
```

This means the controller can send input back into the SteamVR ecosystem even though it is virtual.

2.2.5. Display Redirection

This is one of the most interesting parts of the whole driver. The file is `Desktop/drivers/theia/src/TheiaDisplayRedirectDriver.cpp`, and the class is declared in `Desktop/drivers/theia/src/TheiaDisplayRedirectDriver.h`. This class does two jobs at once. It both implements the SteamVR virtual display interface and captures the rendered backbuffer and prepares it for streaming. In other words, this is the place where Theia starts “seeing” what SteamVR is drawing.

2.2.5.1. How SteamVR Talks to It

SteamVR calls `Present(...)` on the virtual display interface. That is the hook Theia uses to intercept the rendered frame. When `Present(...)` arrives, the driver firstly checks the present information. And there is no problem, it opens the shared backbuffer texture from SteamVR. With that, it copies the backbuffer into an internal capture texture and optionally publishes a preview frame. If everything works correctly, it passes the captured texture to the frame encoder. This is the main render-capture pipeline.

```
void TheiaDisplayRedirectDriver::Present(const vr::PresentInfo_t* present_info,
                                         const std::uint32_t present_info_size) {
    const std::uint64_t count = ++present_count_;
    if (present_info != nullptr && present_info_size >= sizeof(vr::PresentInfo_t)) {
        last_present_frame_id_ = present_info->nFrameId;
        if (count == 1 ||
            (trace_virtual_display_ && trace_interval_ > 0 && (count <= 5 || count % trace_interval_ == 0))) {
            LogPresentSample(*present_info, present_info_size);
        }
        IngestBackbufferTexture(*present_info);
    } else if (count == 1 || (trace_virtual_display_ && trace_interval_ > 0 && count % trace_interval_ == 0)) {
        Log("Theia virtual display Present called with no PresentInfo");
    }
}
```

2.2.5.2. Why D3D11 is Used Here?

The driver uses Direct3D 11 because SteamVR and the desktop rendering pipeline are working with GPU textures. The driver creates a D3D11 device, opens shared textures, and uses the device context to copy resources between textures. That is much more efficient than reading everything back manually from a high-level API.

```
bool TheiaDisplayRedirectDriver::EnsureD3DDevice() {
    if (d3d_device_ != nullptr) {
        return true;
    }

    constexpr D3D_FEATURE_LEVEL kFeatureLevels[] = {
        D3D_FEATURE_LEVEL_11_1,
        D3D_FEATURE_LEVEL_11_0,
        D3D_FEATURE_LEVEL_10_1,
        D3D_FEATURE_LEVEL_10_0,
    };
}
```

2.2.5.3. Preview and Snapshot Behavior

The display redirect driver also supports (on code, normal user does not see most of these) a shared preview frame publisher, optional debug snapshots and trace logging for repeated capture events. The preview path is useful because it lets Theia export a lower-frequency representation of the image while the full encoder handles the actual stream. The debug snapshot path is useful for development, because it allows the driver to save a specific captured frame after a certain number of copies. That helps when diagnosing capture issues.

2.2.6. Frame Encoding

The encoding layer is Desktop/drivers/theia/src/TheiaFrameEncoder.cpp. This is the part that transforms the captured D3D texture into a form that can be sent over the network.

The pipeline is: receive a GPU texture from the display redirect driver / copy it into a staging texture / map it into CPU memory / convert it into NV12 / feed it into a Media Foundation encoder / packetize the encoded output / send it over UDP.

This class is important because it connects graphics capture to network transmission. Without it, the display redirect driver would only collect textures and not actually get them out to the rest of the system.

2.2.6.1. What the Encoder is Doing?

The encoder has many uses. It; limits frame rate so it does not overload the pipe, keeps a small worker queue, supports staged texture copying, converts supported DXGI formats into a CPU-friendly representation, encodes the frame into a streamable format and uses the video stream protocol to split the data into packets. This is a fairly low-level and performance-focused class. It is not a UI-facing component. It exists to keep the stream efficient and predictable.

```
bool TheiaFrameEncoder::CopyTextureToCpuFrame(ID3D11DeviceContext* context,
                                               ID3D11Texture2D* source_texture,
                                               const EncoderFrameInfo& frame_info,
                                               RawFrame& frame) {
    ID3D11Device* device = nullptr;
    source_texture->GetDevice(&device);
    const bool staging_ready = EnsureStagingTexture(device, source_texture);
    SafeRelease(device);
    if (!staging_ready || staging_texture_ == nullptr) {
        return false;
    }
}
```


2.3. Installer

The installer is the packaging and deployment part of Theia. Its job is to take the built desktop application and SteamVR driver payload, guide the user through the required setup steps, and place the files into a usable install directory. Unlike the host or driver, the installer is not part of the runtime stream itself. Instead, it prepares the environment so the host application and driver can run correctly after installation.

This part of the project is implemented as a native Windows installer window using the Win32 API. The installer is intentionally lightweight and direct: it verifies that SteamVR exists, asks the user to confirm the SteamVR location, copies the packaged files, writes the local configuration file, and creates a desktop shortcut.

2.3.1. Installer Entry Point and UI Setup

The main installer logic is in Desktop/installer/src/InstallerMain.cpp.

The installer starts through `wWinMain`, which is the standard Unicode Windows entry point for GUI applications. This function creates the installer state, detects the SteamVR path if possible, registers the window class, creates the installer window, and then runs the standard Windows message loop.

So the installer behaves like a normal Windows desktop setup program:

- it has its own window
- it uses native controls
- it responds to button clicks and text edits
- it runs until the setup flow is complete

The window resources and icon are defined in Desktop/installer/src/TheiaInstaller.rc. This is where the installer gets its branding icon.

```
int WINAPI WinMain(HINSTANCE instance, HINSTANCE previous, LPSTR, int show_command) {  
    return wWinMain(instance, previous, GetCommandLine(), show_command);  
}
```

2.3.2. Installer Flow and Step System

The installer is structured as a simple step-based wizard. The `InstallerStep` enum defines the flow:

- Requirement
- SteamVrPath
- Confirm
- Complete

This makes the installer easy to understand and also easy to present in the report because the logic is very explicit. The UI is updated by `UpdateStepUi(...)`, which shows and hides controls depending on the current step.

2.3.2.1. Step 1: Requirement

The first screen tells the user that SteamVR must already be installed before continuing. This is important because Theia depends on SteamVR for the driver side of the project.

At this stage:

- the installer only shows the requirement message
- the path input is hidden
- the browse button is hidden
- only the Next button is active

This is the gatekeeper step that makes sure the user understands the dependency before installation begins.

```
std::wstring RequirementText() {  
    return L"Theia Desktop Controller kurulumuna devam etmeden once  
    bilgisayarinizda SteamVR yuklu olmalidir.\r\n\r\nDevam etmek icin Next dugmesine basin."  
}
```

2.3.2.2. Step 2: Locate SteamVR

The second screen is where the user selects the SteamVR or Steam folder. The installer tries to detect SteamVR automatically, but it still lets the user browse manually if needed.

This step uses:

- DetectSteamVrRoot()
- NormalizeSteamVrRoot(...)
- BrowseForFolder(...)
- RefreshSteamVrValidity(...)

The installer checks for the two files that prove the folder is valid:

- bin/win64/vrstartup.exe
- bin/win64/vrpathreg.exe

If those files are found, the SteamVR root is considered valid. If the user types or selects a folder and the folder resolves to a valid SteamVR location, the Next button becomes enabled.

```
if (const auto detected = DetectSteamVrRoot(); detected.has_value()) {  
    state.steamvr_root = *detected;  
    state.steamvr_valid = true;  
}
```

This part matters because the installer does not trust the input blindly. It checks the folder structure and makes sure it really points to a SteamVR installation.

2.3.2.3. Step 3: Confirm

Once SteamVR is validated, the installer moves to the confirmation screen. This screen summarizes:

- the SteamVR folder
- the install directory
- the fact that application and driver files will be copied

This is the last checkpoint before writing files to disk. It gives the user a clear preview of what the installer is about to do.

```
std::wstring ConfirmText(const InstallerState& state) {  
    return L"Theia kuruluma hazir.\r\n\r\nSteamVR klasoru:\r\n" + state.steamvr_root.wstring() +  
        L"\r\n\r\nKurulum klasoru:\r\n" + state.install_dir.wstring() +  
        L"\r\n\r\nInstall dugmesine bastiginizda uygulama ve driver dosyalari kopyalanacak."  
}
```

2.3.2.4. Step 4: Complete

After a successful install, the final screen tells the user that Theia Desktop Controller has been installed and can now be launched from the installed folder or desktop shortcut. This is the confirmation stage where the setup flow ends cleanly.

```
case InstallerStep::Complete:  
    SetControlText(window, IdTitle, L"Installation Complete");  
    SetControlText(window,  
        IdBody,  
        L"Theia Desktop Controller was installed successfully.\r\n\r\n  
        You can now launch Theia from the installed folder or desktop shortcut.");
```

2.3.3. SteamVR Detection and Validation

One of the most important helper functions in the installer is DetectSteamVrRoot().

This function tries to find SteamVR automatically by checking common Steam locations such as:

- C:\Program Files (x86)\Steam
- C:\Program Files\Steam
- the custom Steam location used in the project environment

It also checks steamapps/libraryfolders.vdf, so if Steam is installed in another library folder, the installer can still find SteamVR there.

This is a good detail to mention in the report because it shows the installer is not hardcoded to one machine. It tries to be practical and flexible, which is useful when the project is shared or moved between systems.

NormalizeSteamVrRoot(...) makes the selected path smarter by handling cases where the user points to the Steam root instead of the SteamVR folder directly. If needed, it appends the standard steamapps/common/SteamVR path and checks that the required SteamVR executables exist.

```
std::optional<std::filesystem::path> DetectSteamVrRoot() {
    const std::filesystem::path candidates[] = {
        LR"(C:\Program Files (x86)\Steam\steamapps\common\SteamVR)",
        LR"(C:\Program Files\Steam\steamapps\common\SteamVR)",
        LR"(C:\My_Files\Games\Launchers\Steam\steamapps\common\SteamVR)",
    };
};
```

2.3.4. Default Install Path and Package Source

The installer does not build the desktop application or driver from scratch. Instead, the build system prepares a payload for it. This is handled in Desktop/CMakeLists.txt. The CMake logic creates the installer target and then, after the build, prepares this structure: payload/app & payload/driver/theia. Into payload/app, it copies: Theia.exe / the branding folder / theia.ico. Into payload/driver/theia, it copies: the SteamVR driver package. This is the key reason the installer works as a deployment tool rather than a build tool. It installs from a ready-made package.

```
if(THEIA_BUILD_DESKTOP)
    add_dependencies(TheiaInstaller Desktop)
    add_custom_command(TARGET TheiaInstaller POST_BUILD
        COMMAND ${CMAKE_COMMAND} -E make_directory "${TARGET_FILE_DIR:TheiaInstaller}/payload/app"
        COMMAND ${CMAKE_COMMAND} -E make_directory "${TARGET_FILE_DIR:TheiaInstaller}/payload/driver/theia"
        COMMAND ${CMAKE_COMMAND} -E copy_if_different
            "${TARGET_FILE:Desktop}"
            "${TARGET_FILE_DIR:TheiaInstaller}/payload/app/Theia.exe"
        COMMAND ${CMAKE_COMMAND} -E copy_directory
            "${TARGET_FILE_DIR:Desktop}/branding"
            "${TARGET_FILE_DIR:TheiaInstaller}/payload/app/branding"
        COMMAND ${CMAKE_COMMAND} -E copy_if_different
            "${CMAKE_SOURCE_DIR}/assets/branding/theia.ico"
            "${TARGET_FILE_DIR:TheiaInstaller}/payload/app/theia.ico"
        COMMAND ${CMAKE_COMMAND} -E copy_directory
            "${THEIA_INSTALLER_DRIVER_SOURCE_DIR}"
            "${TARGET_FILE_DIR:TheiaInstaller}/payload/driver/theia"
    )
endif()
```

2.3.5. Actual Install Process

The real install work happens in PerformInstall(...). This is the most important function in the installer.

Before copying anything, the installer checks that the payload really exists:

- it looks for payload/app/Theia.exe
- it looks for payload/driver/theia/driver.vrdrivermanifest

If either part is missing, the installer stops and reports an error. That means the install process depends on a valid package layout.

Once the payload is confirmed, the installer creates the target install directory. And then, stops SteamVR processes. Then it copies the app payload into install_dir/app, copies the driver payload into install_dir/steamvr_drivers/theia and writes the desktop configuration file. It lastly creates a desktop shortcut. That is the complete deployment flow.

2.3.5.1. Why It Stops SteamVR First?

StopSteamVrProcesses() is called before copying files. It uses taskkill.exe to stop processes like: vrmonitor.exe / vrserver.exe / vrcompositor.exe / vrdashboard.exe. This is important because SteamVR files and driver files can be locked while the runtime is active. Stopping those processes before copying helps prevent install failures.

```
void StopSteamVrProcesses() {  
    const wchar_t* process_names[] = {  
        L"vrmonitor.exe",  
        L"vrserver.exe",  
        L"vrcompositor.exe",  
        L"vrdashboard.exe",  
    };  
};
```

2.3.5.2. Why It Writes Configuration?

After copying, the installer writes theia_desktop.conf through WriteDesktopConfig(...).

That config includes:

- phone_stream_host=127.0.0.1
- steamvr_root=...
- steamvr_driver_root=...
- stream_port=9944
- stream_fps=60
- stream_bitrate_mbps=20
- stream_udp_payload_bytes=1200

This file is the bridge between the installer and the runtime. The desktop app later reads these values on startup, so the installer is effectively preparing the runtime configuration too.

2.3.6. UI Controls & Event Handling

The installer window is a normal Win32 window with standard controls like title label, body text, path input, browse button, status label, back / next / install / close buttons. These controls are created inside WM_CREATE, and then the window procedure responds to user actions through WM_COMMAND.

```
case WM_CREATE: {  
    auto* create_struct = reinterpret_cast<CREATESTRUCTW*>(l_param);  
    state = reinterpret_cast<InstallerState*>(create_struct->lpCreateParams);  
    SetWindowLongPtrW(window, GWLP_USERDATA, reinterpret_cast<LONG_PTR*>(state));  
  
    HFONT font = static_cast<HFONT*>(GetStockObject(DEFAULT_GUI_FONT));  
  
    CreateWindowW(L"STATIC", L"", WS_CHILD | WS_VISIBLE, 24, 20, 640, 28, window,  
        reinterpret_cast<HMENU*>(IdTitle), nullptr, nullptr);  
    CreateWindowW(L"STATIC", L"", WS_CHILD | WS_VISIBLE | SS_LEFT, 24, 60, 640, 160, window,  
        reinterpret_cast<HMENU*>(IdBody), nullptr, nullptr);  
    CreateWindowW(L"STATIC", L"", WS_CHILD, 24, 210, 140, 22, window,  
        reinterpret_cast<HMENU*>(IdPathLabel), nullptr, nullptr);  
    CreateWindowExW(WS_EX_CLIENTEDGE, L"EDIT", L"", WS_CHILD | ES_AUTOHSCROLL, 24, 236, 540, 26, window,  
        reinterpret_cast<HMENU*>(IdPathEdit), nullptr, nullptr);  
    CreateWindowW(L"BUTTON", L"Browse...", WS_CHILD, 574, 236, 90, 26, window,  
        reinterpret_cast<HMENU*>(IdBrowseButton), nullptr, nullptr);  
    CreateWindowW(L"STATIC", L"", WS_CHILD, 24, 274, 640, 32, window,  
        reinterpret_cast<HMENU*>(IdStatusLabel), nullptr, nullptr);  
    CreateWindowW(L"BUTTON", L"< Back", WS_CHILD | WS_VISIBLE, 448, 372, 96, 30, window,  
        reinterpret_cast<HMENU*>(IdBackButton), nullptr, nullptr);  
    CreateWindowW(L"BUTTON", L"Next >", WS_CHILD | WS_VISIBLE, 560, 372, 104, 30, window,  
        reinterpret_cast<HMENU*>(IdNextButton), nullptr, nullptr);  
    CreateWindowW(L"BUTTON", L"Install", WS_CHILD, 560, 372, 104, 30, window,  
        reinterpret_cast<HMENU*>(IdInstallButton), nullptr, nullptr);  
    CreateWindowW(L"BUTTON", L"Close", WS_CHILD, 560, 372, 104, 30, window,  
        reinterpret_cast<HMENU*>(IdCloseButton), nullptr, nullptr);  
}
```

2.4. Host

The host part of Theia is the desktop controller application that sits between the SteamVR driver and the Android client. This layer is responsible for starting the runtime, receiving the preview stream from the driver, forwarding frames to the mobile device, maintaining the local control window, and showing live status information.

In the overall architecture, the host is the practical bridge between the VR runtime and the network stream. The driver talks to SteamVR, but the host is the application the user actually interacts with. It initializes the desktop environment, keeps the session alive, and exposes the current state of the system through the debug overlay.

2.4.1. Entry Point

The host application starts in Desktop/main.cpp. This file is intentionally small. It creates a `theia::app::Application` object and runs it. On Windows, the file also exposes a `WinMain` entry point so the application can start correctly as a desktop program.

```
#include "App/Application.h"

int main() {
    theia::app::Application application;
    return application.Run();
}

#if defined(_WIN32)
#ifndef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN
#endif
#include <windows.h>

int WINAPI WinMain(HINSTANCE, HINSTANCE, LPSTR, int) {
    return main();
}
#endif
```

Even though this file is short, it is important because it shows that the actual host logic is not hidden in the entry point. Instead, the application behavior is centralized in the `Application` class. That keeps the startup clean and makes the runtime easier to manage. It; creates the main application object, delegates control to the host runtime, and supports both standard `main()` as well as Windows `WinMain()`.

2.4.2. Host Orchestration

The real center of the host side is `Desktop/src/App/Application.cpp`, with the interface declared in `Desktop/src/App/Application.h`. This is the file that ties everything together. It decides when the session starts, how SteamVR is launched, how the frame stream is configured, how the preview receiver is opened, and how the render loop behaves frame by frame.

The host application does not just “show a window”. It coordinates the whole runtime lifecycle which consists of loading configuration, checking dependencies, starting networking, preparing the renderer, opening the preview receiver, pushing frames to the mobile target, updating pose and runtime state, shutting everything down cleanly.

```
class Application {
public:
    Application();
    ~Application();

    int Run();

private:
    void StartHostSession();
    void StopHostSession();
    bool StartSteamVrRuntime();
    void StopSteamVrRuntime();

    renderer::RenderSystem render_system_;
    vr::VrRuntime vr_runtime_;
    tracking::PoseReceiver pose_receiver_;
    streaming::FrameStreamer frame_streamer_;
    std::unique_ptr<streaming::SharedFrameReceiver> preview_receiver_;
    network::UdpSocket telemetry_socket_;
    ui::DebugOverlay debug_overlay_;
    video::VideoPlayer video_player_;
    ViewMode view_mode_ = ViewMode::Video;
    video::VideoStereoLayout video_layout_ = video::VideoStereoLayout::SideBySide;
    video::VideoProjection video_projection_ = video::VideoProjection::Quad;
    bool video_flip_u_ = false;
    bool video_flip_v_ = false;
    bool video_load_requested_ = false;
    std::string video_load_path_;
    std::string steamvr_root_;
    std::string steamvr_driver_root_;
    std::string phone_stream_host_;
    std::string controller_status_ = "Idle";
    streaming::FrameStreamerConfig stream_config_();
    std::uint64_t last_streamed_preview_publish_count_ = 0;
    std::chrono::steady_clock::time_point last_streamed_preview_time_{};
    std::chrono::duration<double> stream_frame_interval_{1.0 / 60.0};
    bool has_last_streamed_preview_time_ = false;
    bool host_session_running_ = false;
    bool start_session_requested_ = false;
    bool stop_session_requested_ = false;
};
```

2.4.2.1. Startup Flow

When `Application::Run()` begins, the host does several setup steps: It initializes WinSock / It logs dependency information / It loads desktop configuration from `theia_desktop.conf` / It resolves the phone stream host / It resolves the SteamVR root directory and driver root / It prepares the stream configuration: port, FPS, bitrate, and payload size / It opens the telemetry socket / It configures the VR runtime / It initializes the render system / It initializes the debug overlay.

This sequence is important because it shows the host is not a passive renderer. It is actively preparing the whole environment before the session begins.

```
core::DependencyProbe::LogSummary(winsock.StartupResult());
const auto desktop_config = LoadDesktopConfig();
stream_config_.port = GetConfigInt(desktop_config, "stream_port", 9944);
stream_config_.target_fps = GetConfigInt(desktop_config, "stream_fps", 60);
stream_config_.bitrate_bps = GetConfigInt(desktop_config, "stream_bitrate_mbps", 20) * 1'000'000;
stream_config_.max_udp_payload_bytes = GetConfigInt(desktop_config, "stream_udp_payload_bytes", 1200);
phone_stream_host_ = ResolvePhoneStreamHost(desktop_config);
steamvr_root_ = ResolveSteamVrRoot(desktop_config);
steamvr_driver_root_ = ResolveSteamVrDriverRoot(desktop_config);
stream_frame_interval_ = std::chrono::duration<double>(1.0 / static_cast<double>(stream_config_.target_fps));

telemetry_socket_.Open(9750);
vr_runtime_.Configure({0.064F, 90.0F});
render_system_.Initialize("Theia Desktop Controller");
vr_runtime_.UpdateEyes(pose_receiver_.LatestPose(), render_system_.StereoAspectRatio());
controller_status_ = host_session_running_ ? "Streaming started" : "Ready";
debug_overlay_.Initialize(render_system_);
debug_overlay_.ShowStartupSummary(render_system_,
                                   vr_runtime_,
                                   frame_streamer_,
                                   pose_receiver_,
                                   telemetry_socket_,
                                   video_player_);

float previous_time = static_cast<float>(glfwGetTime());
```

2.4.2.2. Session Start and Stop

The host has dedicated functions for session control. Some functions [`StartHostSession()` / `StopHostSession()` / `StartSteamVrRuntime()` / `StopSteamVrRuntime()`] are where the desktop side connects to SteamVR and to the driver layer.

When the session starts: Firstly SteamVR is launched or prepared. Then, the driver gets registered and the pose receiver gets activated. After that, the frame streamer is being configured and the shared preview receiver is opened. Lastly, the host enters a running state.

And when the session stops: Firstly, the preview receiver is closed. Then the frame streamer shuts down and the pose receiver stops. Lastly, SteamVR processes are terminated and the driver registration is removed. That cleanup matters because VR runtimes are sensitive to leftover driver state. The host is responsible for leaving the system in a clean condition after each run.

```
void Application::StopHostSession() {
    if (!host_session_running_) {
        return;
    }

    if (preview_receiver_) {
        preview_receiver_->Close();
        preview_receiver_.reset();
    }
    frame_streamer_.Shutdown();
    pose_receiver_.Stop();
    last_streamed_preview_publish_count_ = 0;
    has_last_streamed_preview_time_ = false;
    host_session_running_ = false;
    StopSteamVrRuntime();
    controller_status_ = "Streaming stopped";
    spdlog::info("Desktop host session stopped");
}
```

2.4.2.3. Main Loop Behavior

Inside the main loop, the host keeps doing the same kind of runtime work every frame:

- it polls window events
- it calculates frame time
- it updates pose state
- it updates the preview receiver
- it refreshes the VR runtime
- it renders the current frame
- it draws the debug overlay
- it checks for start/stop requests

```
debug_overlay_.BeginFrame();
debug_overlay_.Render(render_system_,
                      vr_runtime_,
                      frame_streamer_,
                      preview_receiver_.get(),
                      pose_receiver_,
                      telemetry_socket_,
                      video_player_,
                      view_mode_,
                      video_layout_,
                      video_projection_,
                      video_flip_u_,
                      video_flip_v_,
                      video_load_requested_,
                      video_load_path_,
                      host_session_running_,
                      controller_status_,
                      steamvr_root_,
                      steamvr_driver_root_,
                      start_session_requested_,
                      stop_session_requested_);
```

This loop is the heartbeat of the application. It is where the host continuously synchronizes the runtime state with what SteamVR and the mobile client are doing.

2.4.2.4. Host-to-Mobile Frame Forwarding

One of the most important parts of the host is that it does not send raw video blindly. It waits for a new preview frame and also respects the configured frame interval before forwarding the frame to the mobile stream.

```
void Application::StartHostSession() {
    if (host_session_running_) {
        return;
    }

    if (!StartSteamVrRuntime()) {
        return;
    }

    pose_receiver_.Start();
    frame_streamer_.Configure(phone_stream_host_, stream_config_);
    preview_receiver_ = std::make_unique<streaming::SharedFrameReceiver>();
    preview_receiver_->Open();
    last_streamed_preview_publish_count_ = 0;
    has_last_streamed_preview_time_ = false;
    host_session_running_ = true;
    controller_status_ = "Streaming started";
    spdlog::info("Desktop host session started");
}
```

In practice, this means:

- the display redirect driver publishes preview frames
- the host reads them through the shared receiver
- the host forwards them to the mobile side using FrameStreamer
- the stream pacing is controlled by the configured FPS

So the host acts like a traffic manager between the driver and

the phone. It is not the place where frames are created, but it is the place where the stream is handed off to the mobile client.

2.4.3. Renderer

The visual side of the host is handled by Desktop/src/Renderer/RenderSystem.h and Desktop/src/Renderer/RenderSystem.cpp.

This module is responsible for creating the window, initializing OpenGL, building the shader pipelines, creating geometry, and rendering both the local scene and the video surfaces.

2.4.3.1. What This Module Does?

The renderer is built around GLFW and GLAD. It creates the desktop window, sets the OpenGL context, loads the OpenGL function pointers, and then prepares the GPU resources needed by the app.

It has a few main responsibilities: create and manage the desktop window / initialize OpenGL state / compile shader programs / build geometry for scene and video rendering / upload and render video frames / render stereoscopic views for the VR-style scene.

```
public:
    void Initialize(std::string_view window_title);
    void BeginFrame();
    void RenderScene(const vr::VrRuntime& vr_runtime,
                    const video::VideoPlayer& video_player,
                    video::VideoStereoLayout video_layout,
                    video::VideoProjection video_projection,
                    bool flip_u,
                    bool flip_v,
                    bool draw_scene,
                    bool draw_video);
    void RenderVideoFrame(const vr::VrRuntime& vr_runtime,
                          const video::VideoFrame& frame,
                          video::VideoStereoLayout video_layout,
                          video::VideoProjection video_projection,
                          bool flip_u,
                          bool flip_v);
    void EndFrame();
    void PollEvents();
    void RequestClose();
    void Shutdown();
```

2.4.3.2. How the Rendering Pipeline Works?

The code defines two shader pipelines. One for colored scene geometry and the other one for video surfaces. The renderer also prepares different meshes: a cube for the scene, a quad for flat video rendering and sphere and hemisphere geometry for VR-style projection surfaces.

That means the host can render both a general 3D controller environment and the incoming video stream in a way that matches the desired projection mode. But generally SteamVR already does its own geometricization. So, our process only breaks the system more. So, we don't use our geometry and go with the SteamVR's one. Then just render the raw footage.

```
void RenderSystem::InitializeWindow(const std::string_view window_title) {
    if (!glfwInit()) {
        throw std::runtime_error("glfwInit failed");
    }

    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_DOUBLEBUFFER, GLFW_TRUE);

    int window_width = 1280;
    int window_height = 820;
    int window_x = GLFW_DONT_CARE;
    int window_y = GLFW_DONT_CARE;

    if (GLFWmonitor* monitor = glfwGetPrimaryMonitor(); monitor != nullptr) {
        int monitor_x = 0;
        int monitor_y = 0;
        int monitor_width = 0;
        int monitor_height = 0;
        glfwGetMonitorWorkarea(monitor, &monitor_x, &monitor_y, &monitor_width, &monitor_height);
        if (monitor_width > 0 && monitor_height > 0) {
            window_width = std::clamp(static_cast<int>(monitor_width * 0.34F), 1180, 1680);
            window_height = std::clamp(static_cast<int>(monitor_height * 0.46F), 760, 1140);
            window_x = monitor_x + (monitor_width - window_width) / 2;
            window_y = monitor_y + (monitor_height - window_height) / 2;
        }
    }
}
```

2.4.3.3. Video Layout Handling

The renderer understands video layout options like: side-by-side stereo / over-under stereo / flat or spherical projection / UV flipping. Again this was firstly developed because I did not know that SteamVR is already doing it's raw render too. But it helped me understand the structure and it allows me to develop this project futher, beyond SteamVR.

```
if (video_layout == theia::video::VideoStereoLayout::SideBySide) {
    transform.scale_x = 0.5F;
    transform.offset_x = (viewport_x == 0) ? 0.0F : 0.5F;
} else if (video_layout == theia::video::VideoStereoLayout::OverUnder) {
    transform.scale_y = 0.5F;
    transform.offset_y = (viewport_x == 0) ? 0.0F : 0.5F;
}

if (flip_u) {
    transform.offset_x += transform.scale_x;
    transform.scale_x = -transform.scale_x;
}
if (flip_v) {
    transform.offset_y += transform.scale_y;
    transform.scale_y = -transform.scale_y;
}
```

2.4.3.4. Why This File Matters?

This is the visual foundation of the host application. Without it, the host would not have a usable window, a render context, or a way to present the stream. It is also one of the easiest places to show in a screenshot because the file clearly contains the OpenGL initialization and the shader setup.

2.4.4. Debug & Control UI

The user-facing control panel is implemented in Desktop/src/UI/DebugOverlay.h and Desktop/src/UI/DebugOverlay.cpp. This is where the host becomes interactive. The overlay is built with ImGui and acts as the main dashboard for the desktop app.

2.4.4.1. What It Shows?

The overlay is divided into tabs: General / Streaming / Tracking.

Each tab focuses on a different runtime concern. One, session status and paths; other, frame and stream statistics; and last,pose and tracking state.

This is a very useful part of the report because it shows that the host is not only a background service. It is a live controller with real diagnostics.


```

if (ImGui::BeginTabBar("controller-tabs", ImGuiTabBarFlags_NoCloseWithMiddleMouseButton)) {
    if (ImGui::BeginTabItem("General")) {
        active_tab = 0;
        ImGui::EndTabItem();
    }
    if (ImGui::BeginTabItem("Streaming")) {
        active_tab = 1;
        ImGui::EndTabItem();
    }
    if (ImGui::BeginTabItem("Tracking")) {
        active_tab = 2;
        ImGui::EndTabItem();
    }
    ImGui::EndTabBar();
}

```

2.4.4.2. *Why This File Matters?*

The overlay is where you can see the system “thinking”. It exposes the state of the stream, the renderer, the pose data, and the SteamVR setup in one place. That makes it a great screenshot candidate and a strong piece of evidence for the report.

2.4.5. Streaming Handoff

The host-to-mobile network sender is Desktop/src/Streaming/FrameStreamer.cpp, with the interface declared in Desktop/src/Streaming/FrameStreamer.h. This class is responsible for taking a prepared video frame and sending it over UDP to the Android device.

2.4.5.1. *What It Does?*

The streamer has many responsibilities. It stores the target host and port, prepares the UDP socket, ensures an encoder is available, converts the incoming frame into a suitable format, and lastly sends the encoded packet stream to the mobile client.

The important thing here is that the host does not directly hand raw textures to the phone. It encodes the frame first, then sends it as network packets.

```

void FrameStreamer::SendFrame(const video::VideoFrame& frame) {
    if (!IsConfigured() || frame.width <= 0 || frame.height <= 0 || frame.rgb_pixels.empty()) {
        return;
    }
    if (!EnsureSocket() || !EnsureEncoder(frame.width, frame.height)) {
        return;
    }

    const std::uint8_t* source_data[4] = {frame.rgb_pixels.data(), nullptr, nullptr, nullptr};
    const int source_linesize[4] = {frame.width * 4, 0, 0, 0};
    sws_scale(sws_context_,
              source_data,
              source_linesize,
              0,
              frame.height,
              frame->data,
              frame->linesize);
}

```

2.4.5.2. Why It Matters?

This is the part of the host that makes the system cross-device. The desktop app is the sender, and the phone is the receiver. That relationship becomes real in this file.

2.4.6. Pose and Tracking State

```
#include "Tracking/HeadPose.h"

namespace theia::tracking {

class PoseReceiver {
public:
    void Start();
    void Stop();
    void Update(bool move_forward,
               bool move_backward,
               bool move_left,
               bool move_right,
               bool turn_left,
               bool turn_right,
               bool look_up,
               bool look_down,
               float delta_seconds);

    [[nodiscard]] bool IsRunning() const noexcept;
    [[nodiscard]] HeadPose LatestPose() const noexcept;

private:
    bool is_running_ = false;
    HeadPose latest_pose_{{0.0F, 1.6F, 0.0F}, {0.0F, 0.0F, 0.0F}};
};
```

The host-side pose logic is in Desktop/src/Tracking/PoseReceiver.h and Desktop/src/Tracking/PoseReceiver.cpp.

This class stores the current head pose and updates it when movement keys are pressed. It also clamps pitch so the view does not rotate beyond a sane range.

2.4.6.1. What It Does?

The receiver maintains position, yaw, pitch and the roll. Using these it calls the update function. So, The update function supports forward and backward movement, left and right movement, turning left and right and looking up and down.

The host uses this pose state as part of the VR runtime update. In the current architecture, the pose receiver is the host-side representation of movement and orientation state.

```
void PoseReceiver::Update(const bool move_forward,
                          const bool move_backward,
                          const bool move_left,
                          const bool move_right,
                          const bool turn_left,
                          const bool turn_right,
                          const bool look_up,
                          const bool look_down,
                          const float delta_seconds) {

    if (!is_running_) {
        return;
    }
}
```

2.4.6.2. *Why It Matters?*

This part helps explain that the host is not just sending frames. It is also maintaining a live tracking state that feeds the VR runtime and the debug overlay.

3. Mobile

3.1. Project Technologies

3.1.1. Module(s)

- **Android (Android API 24+):**
Project's android side has been designed to run in the Android 13+. It shouldn't have any issues running in Android 7+ devices but there is no device to test it, thus it is considered as uncertain.

3.1.2. Coding Language(s)

- **Kotlin – Android Studio:**
The project is written in Kotlin. It ensures compatibility and optimal performance within Android Studio

3.1.3. Libraries & External Additions

- **androidx.core:core-ktx**
Provides Kotlin-friendly Android core extensions and utility functions.
- **androidx.lifecycle:lifecycle-runtime-ktx**
Provides lifecycle-aware runtime support for the activity and composable flow.
- **androidx.activity:activity-compose**
Used to host the Compose UI inside ComponentActivity.
- **androidx.compose.ui.graphics**
Used for Compose-based color and drawing support.
- **androidx.compose.ui.tooling.preview**
Used for Compose preview and development support.
- **androidx.compose.material3**
Used for the Material 3 design system and text styling.
- **android.media.MediaCodec**
Used for hardware-accelerated H.264 decoding on the device.
- **android.media.MediaFormat**
Used to configure the decoder format and low-latency playback settings.
- **android.view.Surface / SurfaceView**
Used to render the decoded video stream on screen.
- **android.hardware.SensorManager**
Used to access rotation sensors and device motion data.
- **Jetpack Compose BOM**
Used to keep the Compose UI dependencies in sync.

3.2. Application Shell

The entry point of the mobile app is `Mobile/app/src/main/java/com/example/theia/MainActivity.kt`.

This file keeps the UI intentionally minimal. The activity just enables edge-to-edge mode, keeps the screen on, hides the system bars, opens a `SurfaceView` for video rendering, starts or stops the streaming session based on surface lifecycle.

The app also includes a small debug overlay that can be toggled with a double tap. That overlay shows live runtime information such as: receiver state / decoder state / pose state / local IP address / stream and pose ports. It opens with double clicking on screen.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        keepScreenOn()
        enterImmersiveMode()

        setContent {
            TheiaTheme {
                Surface(modifier = Modifier.fillMaxSize(), color = Color.Black) {
                    TheiaStreamScreen()
                }
            }
        }
    }
}
```

This makes the Android app easy to test without needing a separate settings screen.

3.3. Session Coordination

The session manager is `app/src/main/java/com/example/theia/streaming/TheiaStreamSession.kt`

This class is the glue between the three streaming pieces: video receiver / H.264 decoder / pose sender. When the surface becomes available, `start(...)` creates the decoder, then creates the UDP receiver, then starts the pose sender and lastly starts all worker threads. When the surface is destroyed, `stop()` shuts everything down cleanly.

This design keeps the Android side simple because the activity does not need to manage each component directly. The session object owns the lifecycle and keeps the networking/decoding logic together.

```
fun start(surface: Surface) {
    stop()

    val h264Decoder = H264SurfaceDecoder(
        width = videoWidth,
        height = videoHeight,
        surface = surface,
        onStatus = onDecoderStatus,
    )

    val udpReceiver = UdpH264Receiver(
        port = port,
        onSample = h264Decoder::queueSample,
        onStatus = { status ->
            poseSender.updateTargetHost(status.streamHost)
            onReceiverStatus(status)
        },
    )

    decoder = h264Decoder
    receiver = udpReceiver
    h264Decoder.start()
    poseSender.start()
    udpReceiver.start()
}
```

3.4. Video Reception

The incoming video stream is handled by `Mobile/app/src/main/java/com/example/theia/streaming/UdpH264Receiver.kt`. This class listens on UDP port 9944 and rebuilds encoded video samples from packet fragments. It does not assume that each UDP packet contains a full frame. Instead, it parses the custom protocol header, checks packet validity, and groups packets by sample ID until the sample is complete.

```
private fun receiveLoop() {
    try {
        val datagramSocket = DatagramSocket(port).apply {
            receiveBufferSize = 4 * 1024 * 1024
        }
        socket = datagramSocket
        publishStatus( state = "Listening on UDP $port")

        val buffer = ByteArray( size = 1600)
        while (isRunning.get()) {
            val packet = DatagramPacket(buffer, length = buffer.size)
            datagramSocket.receive( p = packet)
            streamHost = packet.address?.hostAddress
            handlePacket(packet.data, packet.length)
        }
    } catch (_: SocketException) {
        if (isRunning.get()) {
            publishStatus( state = "Socket closed unexpectedly")
        }
    } catch (exception: Exception) {
        publishStatus( state = "Receiver error: ${exception.message ?: exception::class.java.simpleName}")
    } finally {
        socket?.close()
        socket = null
    }
}
```

That means the receiver is doing real reconstruction work:

- it validates the custom header
- it ignores invalid packets
- it tracks duplicates and stale packets
- it assembles fragments into a complete encoded sample
- it evicts stale partial samples if they take too long

This is an important part of the project because UDP itself is unreliable, so the client has to reconstruct the stream carefully.

3.5. Video Decoding

The decoding side is `Mobile/app/src/main/java/com/example/theia/streaming/H264SurfaceDecoder.kt`. This class uses Android MediaCodec to decode the incoming H.264 stream directly onto a Surface. That is a good choice for a low-latency viewer because it avoids extra rendering layers and makes the app more efficient on mobile hardware.

The decoder first creates a codec for H.264. Then configures it with low-latency settings so it can feed encoded samples into input buffers. After feeding, drains decoded frames to the surface. Lastly, it keeps a small queue so the stream stays responsive

If the queue gets full, older samples can be dropped. That is a sensible real-time tradeoff because the app is meant to stay current rather than buffer too much outdated video.

```
private fun decodeLoop() {
    try {
        val mediaCodec = MediaCodec.createDecoderByType(MediaFormat.MIMETYPE_VIDEO_AVC)
        val format = MediaFormat.createVideoFormat( mime = MediaFormat.MIMETYPE_VIDEO_AVC, width, height).apply {
            setInteger( name = MediaFormat.KEY_LOW_LATENCY, value = 1)
        }
        mediaCodec.configure(format, surface, crypto = null, flags = 0)
        mediaCodec.start()
        codec = mediaCodec
        publishStatus( state = "Decoder ready")

        val bufferInfo = MediaCodec.BufferInfo()
        while (isRunning.get()) {
            feedInput(mediaCodec)
            drainOutput(mediaCodec, bufferInfo)
        }
    } catch (exception: Exception) {
        publishStatus( state = "Decoder error: ${exception.message ?: exception::class.java.simpleName}")
    } finally {
        codec?.runCatching {
            stop()
            release()
        }
        codec = null
    }
}
```

3.6. Pose Transmission

The pose side is implemented in Mobile/app/src/main/java/com/example/theia/streaming/PhonePoseSender.kt. This class reads motion data from the phone's sensors and sends orientation packets back to the desktop host over UDP. It uses the game rotation vector or rotation vector sensor, depending on what is available on the device.

```
private fun sendLoop() {
    while (isRunning.get()) {
        val packetBytes = latestPacketBytes
        val target = targetAddress
        val currentSocket = socket
        if (packetBytes != null && target != null && currentSocket != null) {
            runCatching {
                val packet = DatagramPacket( buf = packetBytes, length = packetBytes.size, address = target, port)
                currentSocket.send( p = packet)
            }.onFailure { exception ->
                onStatus("Pose send error: ${exception.message ?: exception::class.java.simpleName}")
            }
        }
        Thread.sleep( millis = 8L)
    }
}
```

This is the part that closes the loop between phone and desktop. The mobile app is not only a display target; it also contributes orientation data to the system.

3.7. Protocol Layer

The mobile side also contains the custom protocol definitions:

- Mobile/app/src/main/java/com/example/theia/streaming/VideoStreamProtocol.kt
- Mobile/app/src/main/java/com/example/theia/streaming/PoseStreamProtocol.kt

These files define the packet layout used by the receiver and sender. They make sure both sides agree on magic values, version numbers, header sizes, frame or sequence IDs, and payload size. This is a small but important part of the architecture because the host and mobile code depend on the same packet structure to communicate reliably.

```
val frameId = buffer.long
val sampleId = buffer.long
val packetIndex = buffer.int
val packetCount = buffer.int
val payloadSize = buffer.int
val flags = buffer.int
if (packetCount <= 0 || packetIndex < 0 || packetIndex >= packetCount || payloadSize < 0) {
    return null
}
if (HeaderSize + payloadSize > length) {
    return null
}

return Header(
    frameId = frameId,
    sampleId = sampleId,
    packetIndex = packetIndex,
    packetCount = packetCount,
    payloadSize = payloadSize,
    flags = flags,
)
```

4. Project Analysis

4.1. Test

The tests were made on my personal computer and phone. The system is tested on up to 30-minute time periods. The lag and packet loss were low. So, we can say that all tests were successful.

4.2. Upgradeable Things

Now, the Theia only works as attached to SteamVR and does not work individually. In the future may be we can develop this project to be independent of any platforms and provide solid work for users.

5. Project Comments

5.1. To-Do's When Using

Do not forget to configure the desktop app according to IP of the phone. Which can be found by double clicking and opening the debug UI on mobile app.

5.2. Creator Comments

This was my first Orange project idea. Janus was possible because I wanted a VR bu had no money. So, yeah. I love this project.

Hooah

-Famura

6. Credits

- Famura Only Project

7. References

- SteamVR
Valve Corporation. (n.d.). *SteamVR*. <https://store.steampowered.com/steamvr>
- OpenVR
Valve Corporation. (n.d.). *OpenVR SDK*. <https://github.com/ValveSoftware/openvr>
- FFmpeg
FFmpeg Developers. (n.d.). *FFmpeg*. <https://ffmpeg.org/>
- x264
VideoLAN. (n.d.). *x264 H.264 encoder*. <https://www.videolan.org/developers/x264.html>
- Media Foundation
Microsoft. (n.d.). *Microsoft Media Foundation*. <https://learn.microsoft.com/en-us/windows/win32/medfound/microsoft-media-foundation>
- OpenGL
Khronos Group. (n.d.). *OpenGL*. <https://www.opengl.org/>
- GLFW
GLFW. (n.d.). *GLFW*. <https://www.glfw.org/>
- GLAD
GLAD. (n.d.). *GLAD OpenGL loader*. <https://glad.dav1d.de/>
- GLM
G-Truc Creation. (n.d.). *GLM (OpenGL Mathematics)*. <https://github.com/g-truc/glm>
- Direct3D 11
Microsoft. (n.d.). *Direct3D 11*. <https://learn.microsoft.com/en-us/windows/win32/direct3d11/direct3d-11-graphics>
- Android Studio
Google. (n.d.). *Android Studio*. <https://developer.android.com/studio>
- Android MediaCodec
Google. (n.d.). *MediaCodec API*. <https://developer.android.com/reference/android/media/MediaCodec>
- Android SensorManager
Google. (n.d.). *SensorManager*. <https://developer.android.com/reference/android/hardware/SensorManager>
- Jetpack Compose
Google. (n.d.). *Jetpack Compose*. <https://developer.android.com/jetpack/compose>

- CMake
Kitware. (n.d.). *CMake*. <https://cmake.org/>
- Dear ImGui
Ocornut. (n.d.). *Dear ImGui*. <https://github.com/ocornut/imgui>
- spdlog
Gabi Melman. (n.d.). *spdlog*. <https://github.com/gabime/spdlog>
- stb
Barrett, S. (n.d.). *stb libraries*. <https://github.com/nothings/stb>
- Windows API
Microsoft. (n.d.). *Windows API*. <https://learn.microsoft.com/en-us/windows/win32/api>
- Win32 API
Microsoft. (n.d.). *Win32 API*. <https://learn.microsoft.com/en-us/windows/win32/>
- H.264
ITU-T. (2019). *Advanced video coding for generic audiovisual services (H.264)*.
- UDP
Postel, J. (1980). *User Datagram Protocol*. RFC 768. <https://doi.org/10.17487/RFC0768>
- Winsock
Microsoft. (n.d.). *Windows Sockets 2 (Winsock)*. <https://learn.microsoft.com/en-us/windows/win32/winsock/windows-sockets-start-page>

8. Disk Contents

- This report on .docx format
- This report on .pdf format
- Source code of the project
- Android Release of the project
- Desktop Release of the project
- Igra Rock'en'Roll Cela Jugoslavia by Elektricini Orgazam